

INPUT (STDIN)

6 7

1 2

1 3

2 6

3 4

3 5

4 6

5 6

EXPECTED OUTPUT

2

4

1 3 5 6

3

1 2 6

INPUT (STDIN)

6 7

1 4

3 5

4 6

5 6

2 2

1 3

2 6

EXPECTED OUTPUT

2

4

1 3 5 6

3

1 4 6

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int bfs(int n,int s,int t)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

920

Test Case 3

NLOC

110

Graph9.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 2005
7
8 int head[MAXN], to[MAXM * 2], cap[MAXM * 2], flow[MAXM * 2], nxt[MAXM * 2], edge_cnt;
9 int parent_edge[MAXN];
10 int parent_node[MAXN];
11
12 void add_edge(int u, int v) {
13     to[edge_cnt] = v;
14     cap[edge_cnt] = 1;
15     flow[edge_cnt] = 0;
16     nxt[edge_cnt] = head[u];
17     head[u] = edge_cnt++;
18
19     to[edge_cnt] = u;
20     cap[edge_cnt] = 0;
21     flow[edge_cnt] = 0;
22     nxt[edge_cnt] = head[v];
23     head[v] = edge_cnt++;
24 }
25
```

```

26 int bfs(int n, int s, int t) {
27     int q[MAXN], front = 0, rear = 0;
28     int visited[MAXN];
29     memset(visited, 0, sizeof(visited));
30     memset(parent_edge, -1, sizeof(parent_edge));
31
32     q[rear++] = s;
33     visited[s] = 1;
34
35     while (front < rear) {
36         int u = q[front++];
37         if (u == t) return 1;
38
39         for (int e = head[u]; e != -1; e = nxt[e]) {
40             int v = to[e];
41             if (!visited[v] && cap[e] - flow[e] > 0) {
42                 visited[v] = 1;

```

```

43         parent_edge[v] = e;
44         parent_node[v] = u;
45         q[rear++] = v;
46     }
47 }
48 }
49 return 0;
50 }

```

```
51
```

```
52 int max_flow(int n, int s, int t) {
```

```
53     int total_flow = 0;
```

```
54     while (bfs(n, s, t)) {
```

```
55         for (int v = t; v != s; v = parent_node[v]) {
```

```
56             int e = parent_edge[v];
```

```
57             flow[e] += 1;
```

```
58             flow[e ^ 1] -= 1;
```

```
59         }
```

```
60     total_flow++;
61 }
62 return total_flow;
63}
64
65int path_nodes[MAXN];
66
67int main() {
68     int n, m;
69     if (scanf("%d %d", &n, &m) != 2) return 0;
70
71     memset(head, -1, sizeof(head));
72     edge_cnt = 0;
73
74     for (int i = 0; i < m; i++) {
75         int u, v;
```

```
76     scanf("%d %d", &u, &v);
77     add_edge(u, v);
78 }
79
80 int k = max_flow(n, 1, n);
81 printf("%d\n", k);
82
83 for (int i = 0; i < k; i++) {
84     int curr = 1;
85     int path_len = 0;
86     path_nodes[path_len++] = curr;
87
88     while (curr != n) {
89         for (int e = head[curr]; e != -1; e = nxt[e]) {
90             if (flow[e] > 0) {
91                 flow[e] = 0;
```

```

92         curr = to[e];
93         path_nodes[path_len++] = curr;
94         break;
95     }
96 }
97 }
98
99 printf("%d\n", path_len);
100 for (int j = 0; j < path_len; j++) {
101     printf("%d%c", path_nodes[j], j == path_len - 1 ? '\n' : ' ');
102 }
103 }
104
105 return 0;
106 }
107

```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
6 7
```

```
1 2
```

```
1 3
```

```
2 6
```

```
3 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph9.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

```
3
```

```
1 2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6 7
```

```
1 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
2 2
```

```
1 3
```

```
2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph9.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

3

1 4 6

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```